

Project Configuration Management Plan

furlink

Asia Pacific College
Humabon Place, Magallanes Village
Makati City, Metro Manila, 1232

May 2026

Table of Contents

Introduction.....	1
Roles and Responsibilities.....	1
Configuration Control	2
Configuration Management Database (CMDB)	4
Configuration Status Accounting	4
Configuration Audits.....	5

Introduction

The purpose of the Configuration Management (CM) Plan is to establish and describe the systematic framework for conducting configuration management throughout the entire project lifecycle of furlink. By defining a structured, coordinated approach, this document details how configuration items are tracked, implemented, verified, and communicated to project stakeholders, thereby ensuring that all system builds, database schema trees, and technical documentation remain consistent and accurate. In the absence of a formalized configuration management plan, software and information technology initiatives are highly vulnerable to version conflicts, omitted requirements, or redundant work driven by a lack of strict baseline and change controls. Given that furlink is an advanced digital booking ecosystem designed to replace informal pet care arrangements with a structured, cloud-native marketplace, maintaining structural integrity across parallel development lines is paramount. This necessity is further amplified by the inherent technical complexity of integrating a modern serverless architecture composed of React.js, Supabase, Vercel compute nodes, the PayMongo transactional gateway, Gmail SMTP communication relays, and the Google Gemini 2.5 Flash API. Consequently, this plan acts as the authoritative strategy for preventing architectural divergence across sandbox testing, staging, and production environments, while explicitly establishing the roles of the Configuration Control Board (CCB) and defining the maintenance procedures for the platform's digital assets.

Roles and Responsibilities

The furlink Configuration Control Board (CCB) functions as a unified governing body composed of the entire core project team. Because the platform utilizes a flat startup layout with modular development boundaries, all members are responsible for evaluating the structural and architectural feasibility of proposed infrastructure transformations.

- **Lalaine Andaya (Project Manager):** The furlink Configuration Control Board (CCB) functions as a unified governing body composed of the entire core project team. Because the platform utilizes a flat startup layout with modular development

boundaries, all members are responsible for evaluating the structural and architectural feasibility of proposed infrastructure transformations.

- **Atasha Frances Gayle Doria (Lead Full-Stack Developer):** Holds ultimate accountability for platform architecture, serverless infrastructure synchronization, and repository integrity. She is responsible for assigning version identifiers, managing branch permissions, configuring environment variables across production hosts, and holding final tie-breaking authority over code-level technical implementation decisions.
- **Michelle Reina Pineda (Quality Assurance):** Responsible for drafting formal change requests when module telemetry or user feedback requires adjustments to the software baseline. She ensures all locally engineered features comply with repository standards and evaluates the side-effect impacts of stack updates on neighbor dependencies.
- **Feliz Angelica Salting (Full-Stack Developer):** Formulates drafts comprehensive CCRs when user or provider feedback necessitates adjustments to active feature baselines. Responsible for assessing cross-modular technical impacts during board evaluations, ensuring that approved code alterations strictly follow established repository templates and standards.

Configuration Control

Configuration Control for furlink is the process of systematically evaluating, controlling, and managing all software framework, database schema, and cloud infrastructure modifications throughout the project lifecycle. To prevent processing bottlenecks while maintaining system stability, change request are classified into two strict categories based on their technical impact. Major Changes refer to any modifications that alter the core project scope, timeline, database schema structures, primary API endpoints—such as migrating from Google Gemini to an alternative language engine—or budget parameters. These high-impact requests must strictly undergo the full, formal Change Control Board evaluation loop and require unanimous consensus from the team to proceed. Conversely, Minor Changes

encompass low-risk, low-impact development adjustments that do not alter the core architecture, modify database columns, or jeopardize the project baseline. These include simple user interface alignment fixes, text revisions, or minor dependency patch updates. Minor changes bypass formal board routing entirely and can be mutually cleared by the Project Manager and Lead Developer as simple tasks during standard sprint backlogs.

The end-to-end infrastructure change workflow begins with identification, where technical bugs, stack deprecations, or performance shortfalls are captured and documented within OpenProject as formal tickets. Next, during the Preliminary Analysis phase, the Project Manager reviews the ticket to evaluate its scope and cost impact, formally classifying the submission as either a Major or a Minor Change. If the request is classified as a Minor Change, it bypasses the formal board assessment and moves directly into sprint allocation; however, if it is classified as a Major Change, it is formally scheduled for a board review session. During the Board Evaluation phase, the entire project team reviews the proposal against two critical criteria: its explicit alignment with the platform's market goals and its technical feasibility within the current cloud-native tech stack. Denied requests are logged but may be updated with newer data and re-submitted later for board reconsideration.

Once a major change request is approved by the board, the lifecycle transitions into Implementation and Isolation, where the Project Manager re-baselines the project documentation and scope metrics, while the Lead Developer simultaneously authorizes the creation of a dedicated Feature Branch in GitHub to isolate the infrastructure configuration update away from production code. During active development, any resulting technical blockers or configuration anomalies are reported transparently during the team's daily stand-up meetings. Following the completion of code updates, the project undergoes Testing and Verification, an essential quality assurance check where comprehensive software and regression testing are executed within an isolated staging environment, allowing the Quality Assurance to validate that the new infrastructure components remain perfectly stable. Finally, the process reaches Closure once all features are fully verified; the change request ticket is officially closed within OpenProject, and the Lead Full-Stack

Developer safely merges the staging codebase into the master development branch for automated, live cloud deployment.

Configuration Management Database (CMDB)

For the furlink platform, the CMDB is structured as a coordinated ecosystem of three specialized cloud tools to maintain a complete history of project assets. First, GitHub acts as the source code and database migration schema registry, providing a complete, auditable history of every code adjustment alongside its author, timestamp, and commit description. Second, Microsoft 365 manages project documentation through a tiered strategy, utilizing Microsoft tools as the primary hub for live document co-authoring and SharePoint as the shared workspace for finalized, Read-Only project artifacts mirrored into the academic repository. Third, Vercel and Supabase coordinate to manage live serverless computing infrastructure, data configurations, and system environment variables. Vercel acts as the host environment variable registry by dynamically injecting encrypted API credentials—such as the PayMongo authentication secrets, GMAIL SMTP application keys, and Google Gemini connection string tokens—at the cloud edge. Simultaneously, supase serves as the repository for the relational PostgreSQL database schema structures, tracking SQL tables and row-level security policies to allow the Lead Full-Stack Developer to handle swift rollbacks if database discrepancies are identified.

Configuration Status Accounting

Configuration Status Accounting for furlink involves the continuous collection and reporting of the status of all CIs to ensure the team and stakeholders have real-time visibility into the project's progress. To track and account for active infrastructure assets, the team utilizes a combination of digital tracking systems and synchronous sprint checkpoints. OpenProject serves as the primary status accounting database, where individual team members are responsible for updating the status logs of their assigned work packages to accurately reflect its state. Complementing this platform, the team conducts stand-up meetings to

synchronize development trajectories and instantly flags any open configuration change requests (CCRs) acting as code or architectural blockers.

To satisfy reporting requirements, the Project Manager compiles and submits comprehensive status summaries during scheduled milestones. These evaluations systematically report on change request metrics, including aging distributions to track how long infrastructure changes remained open, and trending maps to identify which specific architectural modules are undergoing the most frequent code adjustments. Additionally, status accounting reports detail version control metrics across software frames, data structures, and supporting documentation, while providing build logs that detail file states, incorporated changes and active CI relationships. Prior to any formal product release, the Lead Full-Stack Developer works closely with the Project Manager to verify that all system dependencies are updated to their correct release numbers, while preparing formal configuration summaries to be presented to panelists for high-level project validation during scheduled defense windows.

Configuration Audits

Configuration audits will occur as an ongoing pre-release check prior to any major software launch or at the discretion of the Project Manager and project advisers. All configuration audits are performed directly by the Lead Full-Stack Developer in close coordination with the Project Manager, ensuring that all code alterations and deployment procedures follow the guidelines established in this plan.

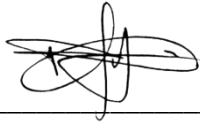
The audit workflow requires the execution of six distinct technical evaluation tasks:

1. The Lead Full-Stack Developer establishes an audit environment by spinning up an isolated, temporary clone staging sandbox within Vercel to inspect modifications without altering live system traffic.
2. The Lead Full-Stack Developer copies the latest software builds, database schemas, and documentation files into this audit environment for holistic review.

3. The Lead Full-Stack Developer verifies versioning and numbering conventions, ensuring that all code dependencies in package.json are anchored to immutable version states according to the defined branching strategy
4. The Lead Full-Stack Developer conducts a historical analysis by cross-referencing timestamps and commit descriptions in GitHub and OpenProject to ensure all baseline modifications were properly recorded.
5. The Lead Full-Stack Developer, in coordination with the Quality Assurance, executes testing on the staging build to confirm that all functional requirements are perfectly met.
6. The Lead Full-Stack Developer conducts a comprehensive artifact review to ensure all system secrets are securely partitioned and that all approved CCRs have been fully incorporated and logged in the system records.

Sponsor Acceptance

Approved by the Project Sponsor:



Date: May 20, 2026

Manuel R. Doria Jr.

Project Sponsor Representative